System for Artificial Intelligence

NPU Architecture & Ascend Programming

Siyuan Feng
Shanghai Innovation Institute

Recap: Overview of Machine Learning Systems

上海创智学院
Shanghai Innovation Institute



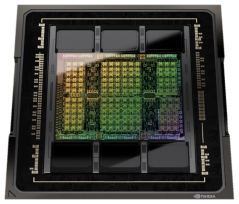
ML Models

Automatic Differentiation

Graph Optimization

Parallelism / Distributed

Hardware Acceleration



NVIDIA GPU



HUAWEI NPU



Mobile devices

This Lecture

Hardware Changes the Execution Plan

上海创智学院
Shanghai Innovation Institute

$$Y = \mathrm{LeakyReLU}(AB)$$

Same workload: matrix multiplication followed by element-wise activation

GPU: thread and warp organization

Blocks / warps → Matmul + activation

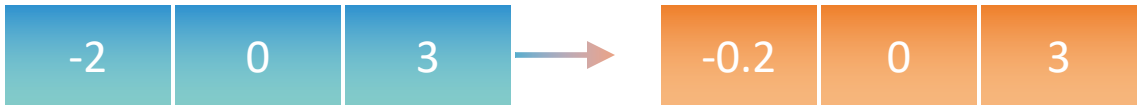
Registers and shared memory

NPU: cooperating execution units

Cube: matmul → Vector: activation

L1 / L0C and Unified Buffer

Activation example α = 0.1



OUTLINE

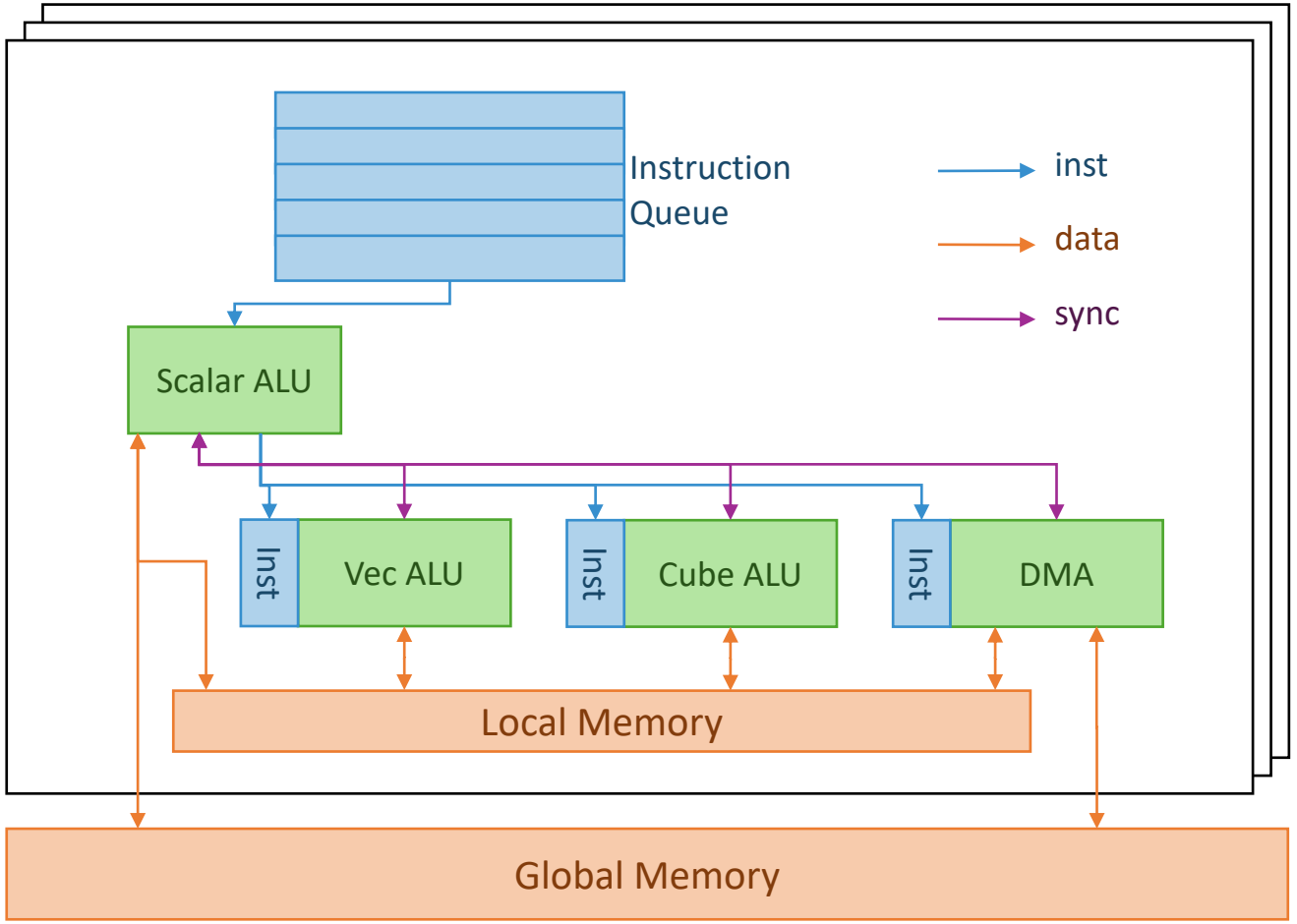
01 HUAWEI NPU Architectures

02 Ascend C Programming

01

HUAWEI NPU Architectures

NPU AI Core Architecture



Scalar Unit:

```
c = a + b
```

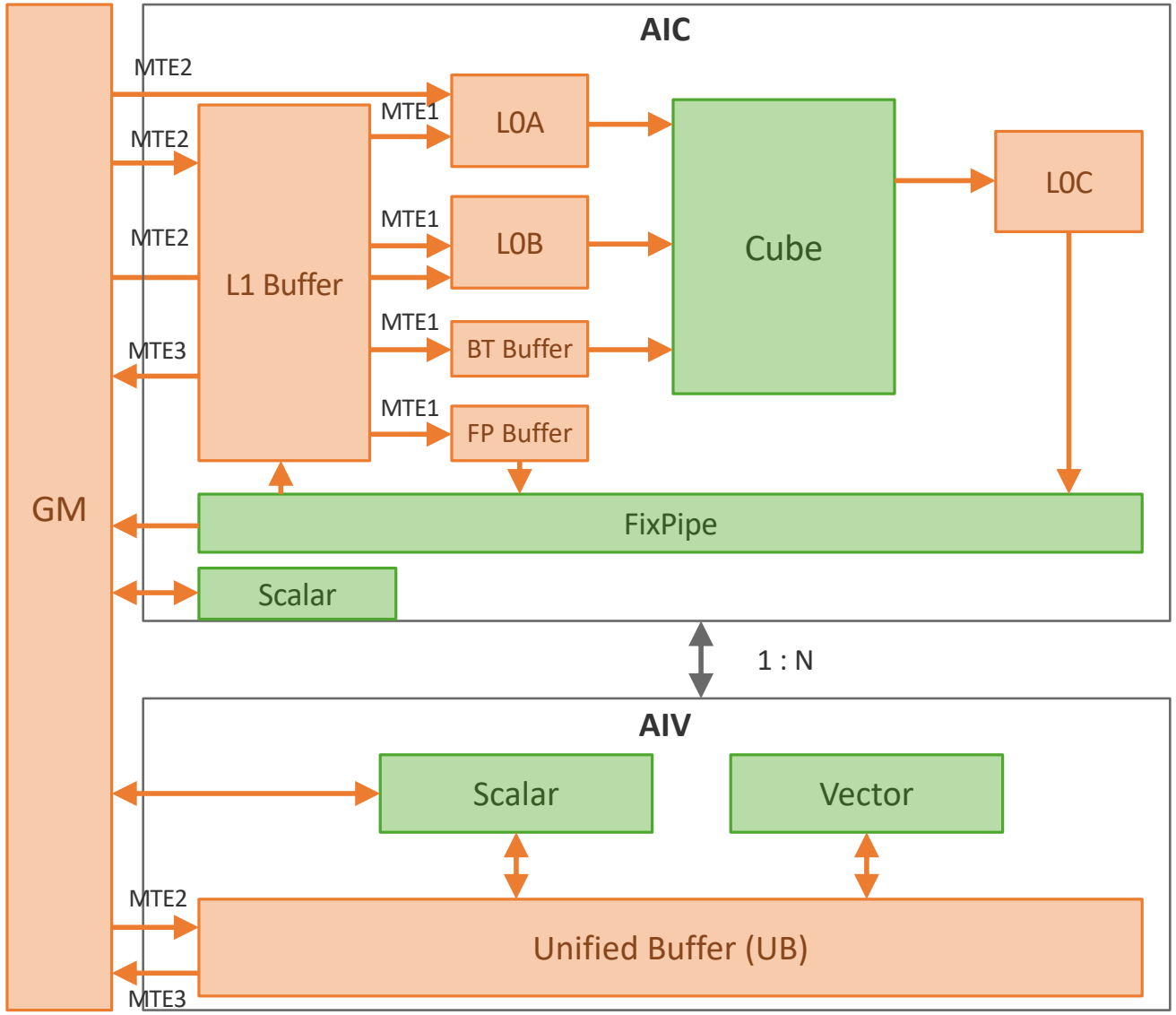
Vector Unit:

```
C[0:1024] = A[0:1024] + B[0:1024]
```

Cube Unit:

```
C[0:16, 0:16] +=
    A[0:16, 0:16] * B[0:16, 0:16]
```

Disaggregated AI Core Architecture (910 B / C)



- Multi-level Memory Scope
- **Not "full-mesh"** access across ALU and memory

Compute / transfer units

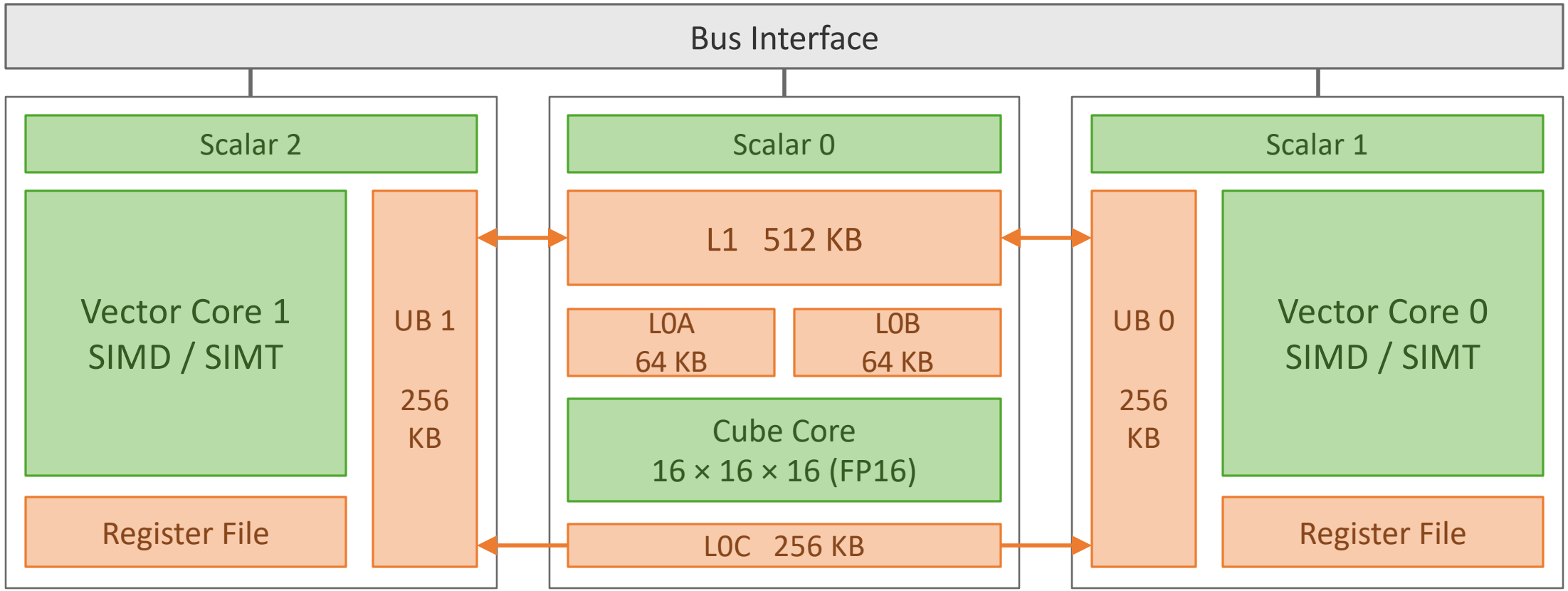
Memory / buffers

Data movement

950 Pairs One Cube with Two Vectors

上海创智学院
Shanghai Innovation Institute

One AI subsystem. Physical organization and selected C/V links



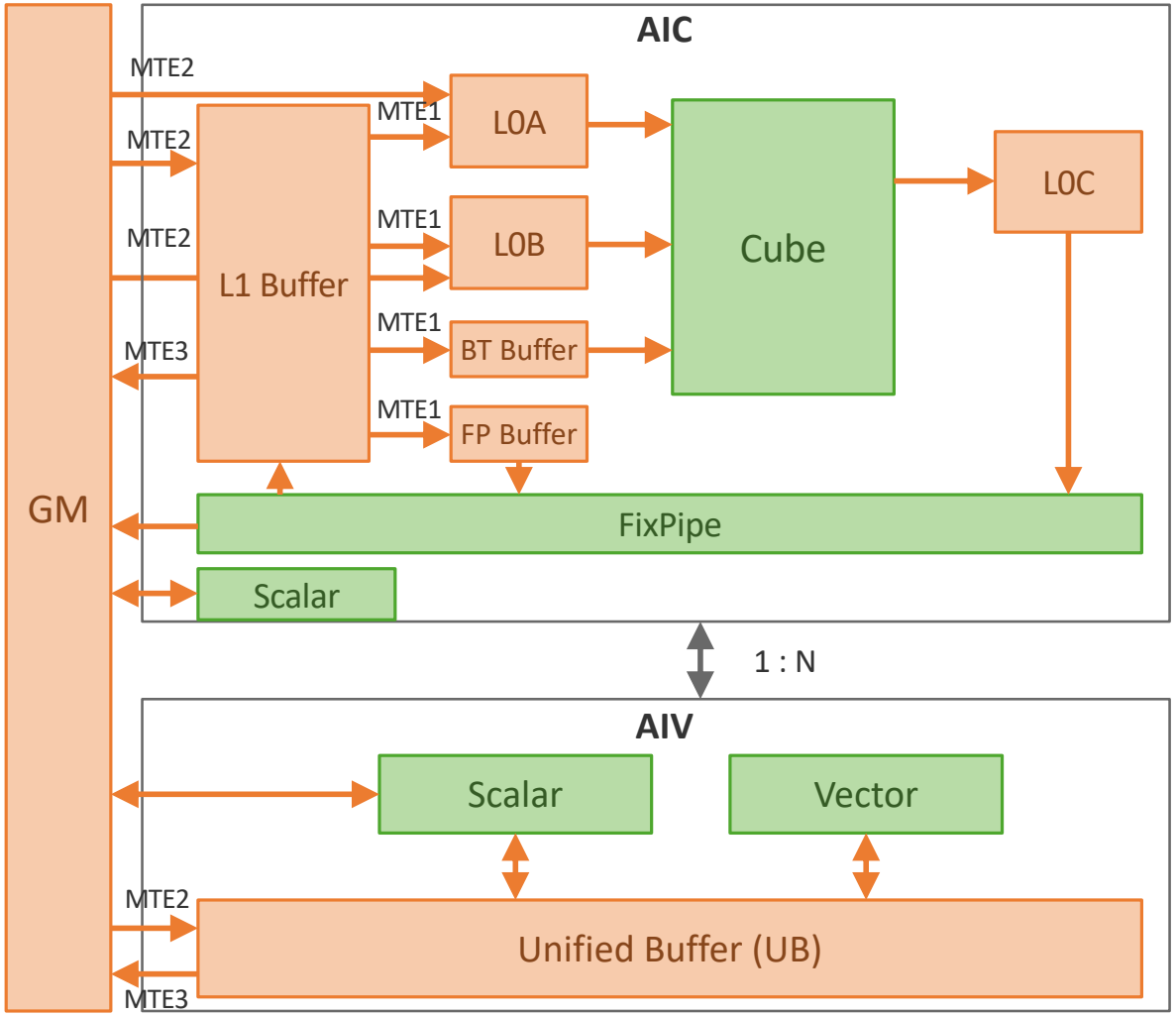
950PR and 950DT: Memory Tradeoffs

上海创智学院
Shanghai Innovation Institute

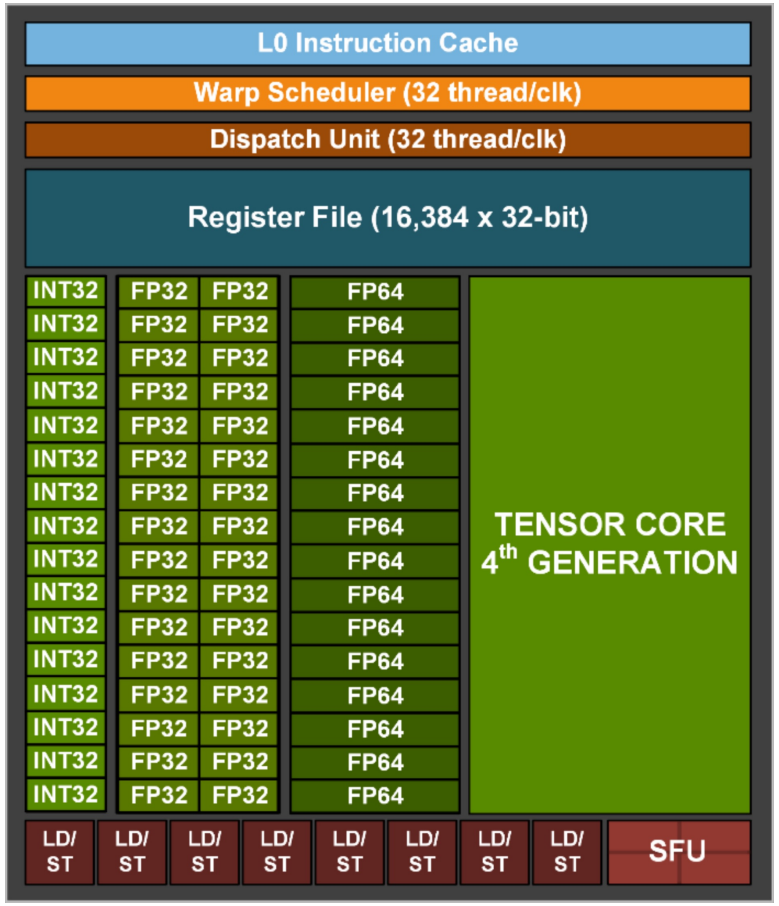
Published upper bounds; individual device configurations vary

Specification	950PR	950DT
Cube / Vector cores	Up to 32 / 64	Up to 36 / 72
Memory capacity	Up to 128 GB	Up to 144 GB
Memory bandwidth	Up to 1.6 TB/s	Up to 4 TB/s
Named workloads	Prefill, recommendation	Decode, training

Discussion: comparison between NPU and GPU



HUAWEI NPU Architecture



NVIDIA GPU Architecture

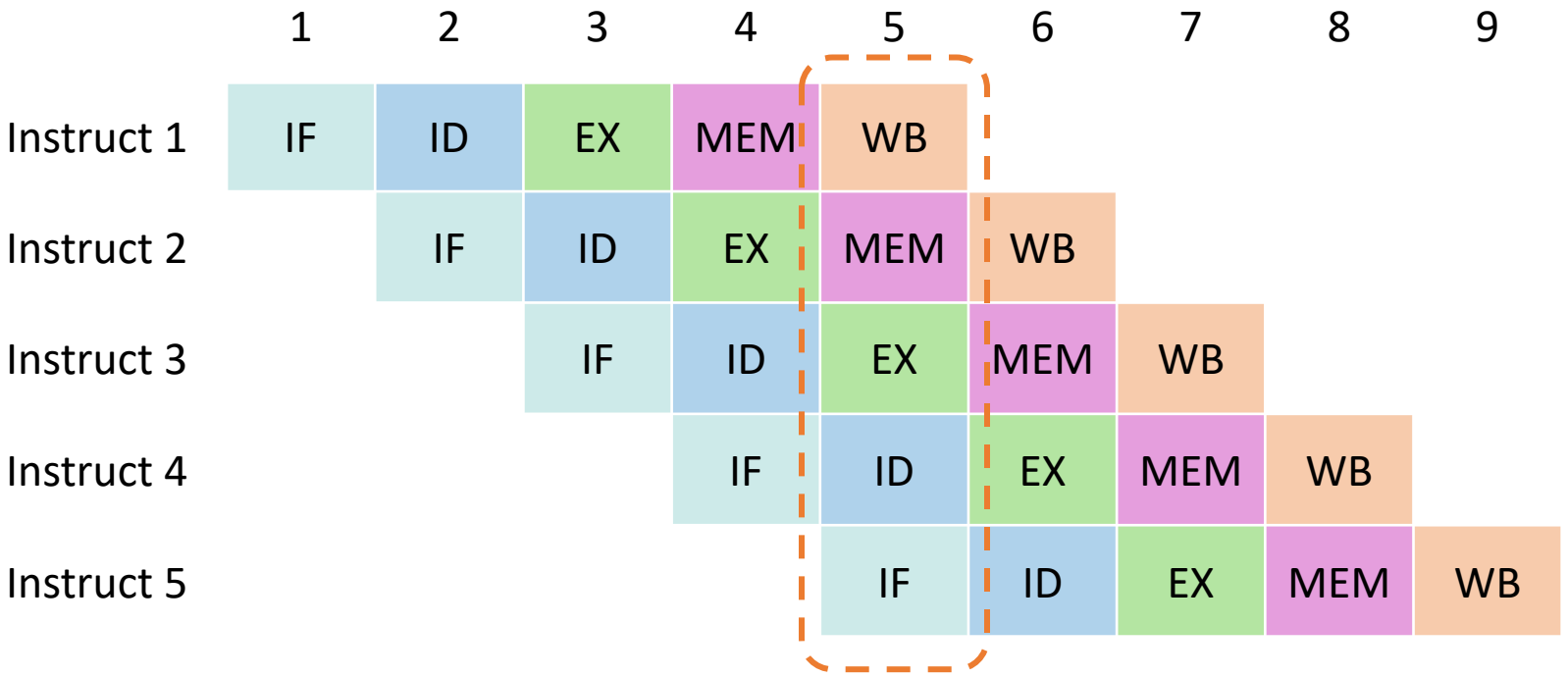
02

Ascend C Programming

Recap: Five-Stage Pipeline in CPU

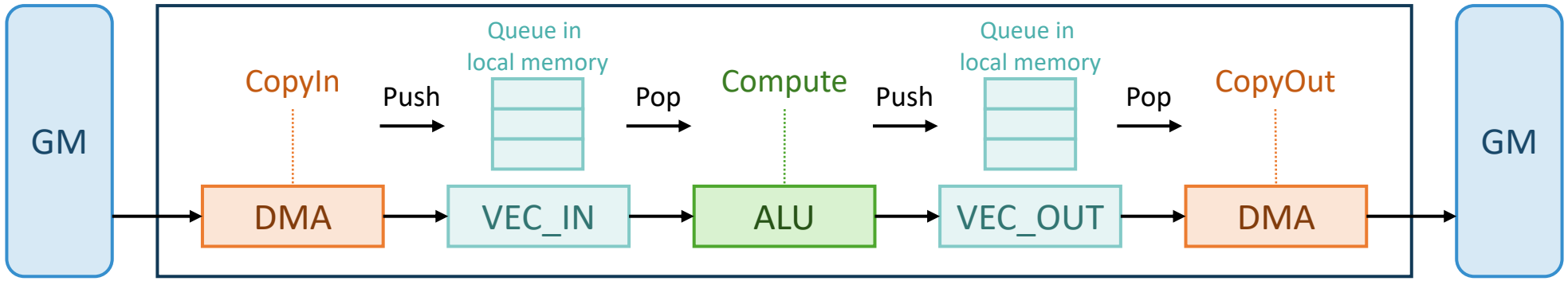
上海创智学院
Shanghai Innovation Institute





Discussion: Why?

Ascend C Programming for AIV



Three tasks:

1. **CopyIn:** load data from global memory to local memory and push to queue VEC_IN
2. **Compute:** load from queue VEC_IN, do computation, and then store to VEC_OUT
3. **CopyOut:** read data from queue VEC_OUT, and write the result to global memory

Vector Addition Splits into Tiles

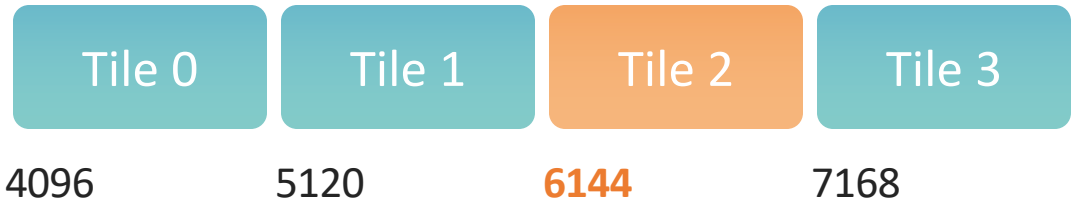
上海创智学院
Shanghai Innovation Institute

FP32 N = 8192 2 work partitions 1024 elements per tile

All elements

Core 0: elements 0–4095

Core 1: elements 4096–8191



Each core processes four tiles

Each tile reads x and y, then writes z

$$\text{offset} = \text{core_id} \times 4096 + \text{tile_id} \times 1024$$

$$\text{core_id} = 1, \text{tile_id} = 2: \text{offset} = 6144$$

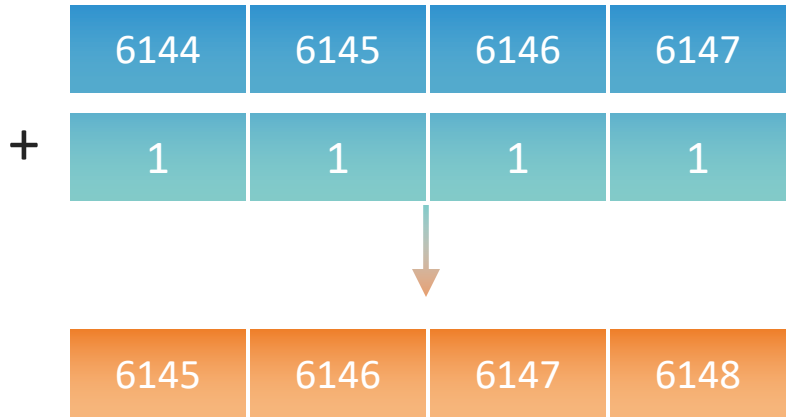
A Tile Moves Through Three Queues

上海创智学院 Shanghai Innovation Institute

Ascend C member-function excerpt TILE = 1024 FP32 elements

```
__aicore__ inline void Compute() {
  auto x = xQueue.DeQue<float>();
  auto y = yQueue.DeQue<float>();
  auto z = zQueue.AllocTensor<float>();
  AscendC::Add(z, x, y, TILE);
  zQueue.EnQue(z);
  xQueue.FreeTensor(x);
  yQueue.FreeTensor(y);
}
```

First four values of this tile





Pseudocode for AIV Programming

上海创智学院
Shanghai Innovation Institute

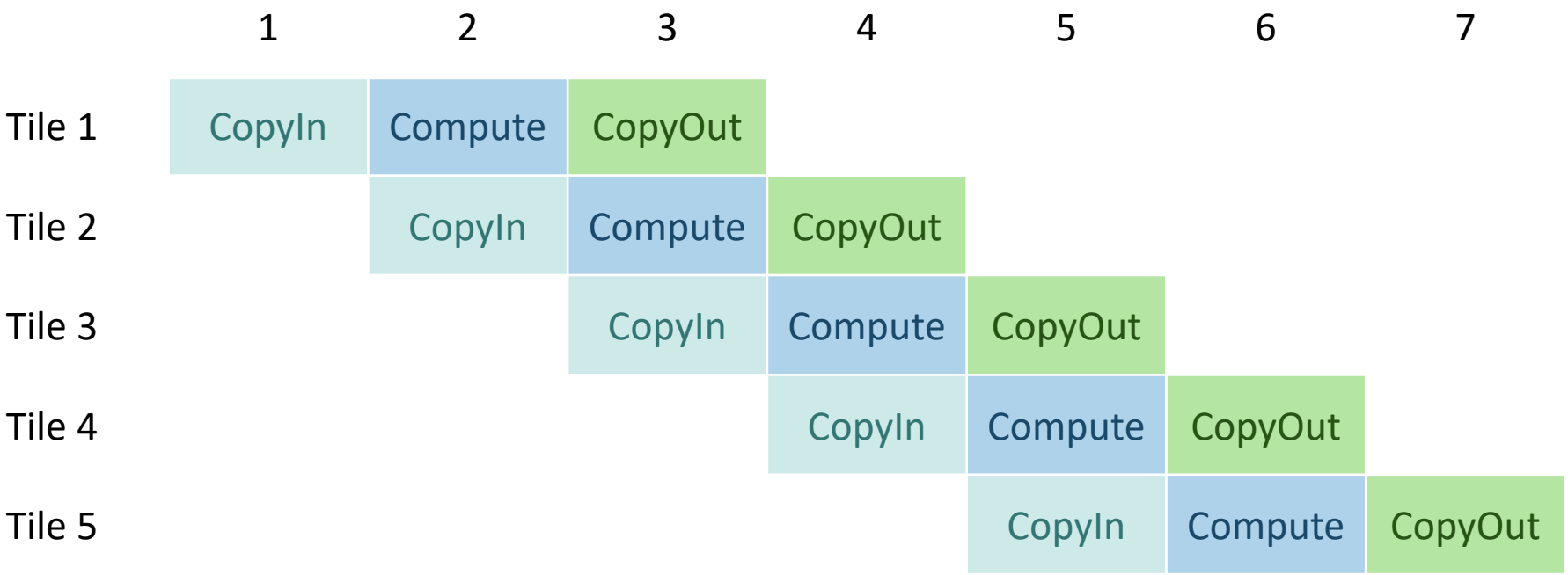
```
TQue<VecIn, 1> queIn;
TQue<VecOut, 1> queOut;

for-loop {
  // CopyIn Stage
  auto tensor = queIn.AllocTensor<half>();        // Allocate tensor at local for input
  DataCopy(tensor, gm, len);                      // Copy data from gm to local
  queIn.EnQue(tensor);                            // push tensor to queIn after load

  // Compute Stage
  auto tensor = queIn.DeQue<half>();              // Get tensor from queIn when ready
  auto tensorOut = queOut.AllocTensor<half>();    // Allocate tensor at local for output
  OP(tensorOut, tensor, len);                    // Main computation
  queIn.FreeTensor(tensor);                       // Free input tensor at local
  queOut.EnQue(tensorOut);                        // Push output tensor to queOut

  // CopyOut Stage
  auto tensor = queOut.DeQue<half>();             // Get tensor from queOut when ready
  DataCopy(gmOut, tensor, len);                  // Copy data from local to gm
  queOut.FreeTensor(tensor);                      // Free output tensor at local
}
```

Pipeline of AIV

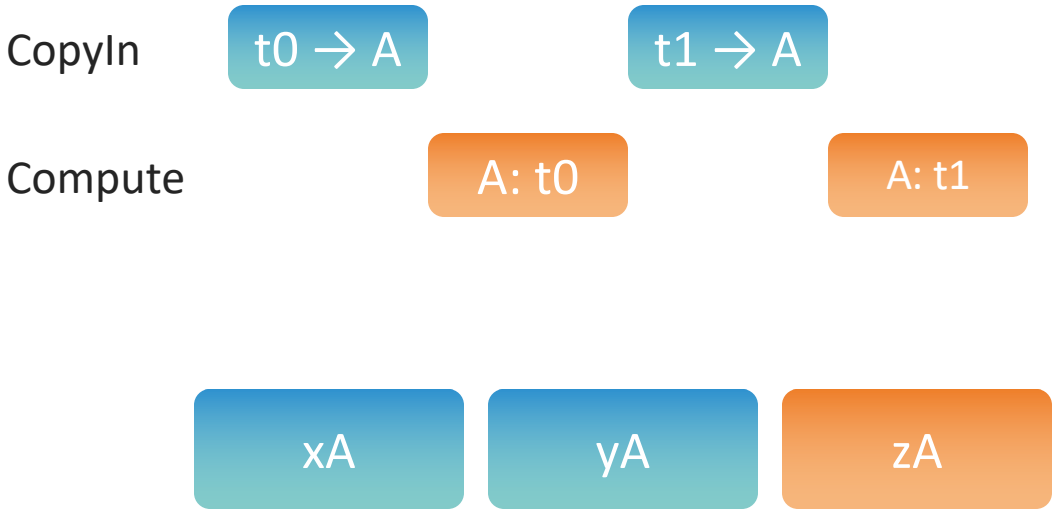


Two Buffers Enable Overlap

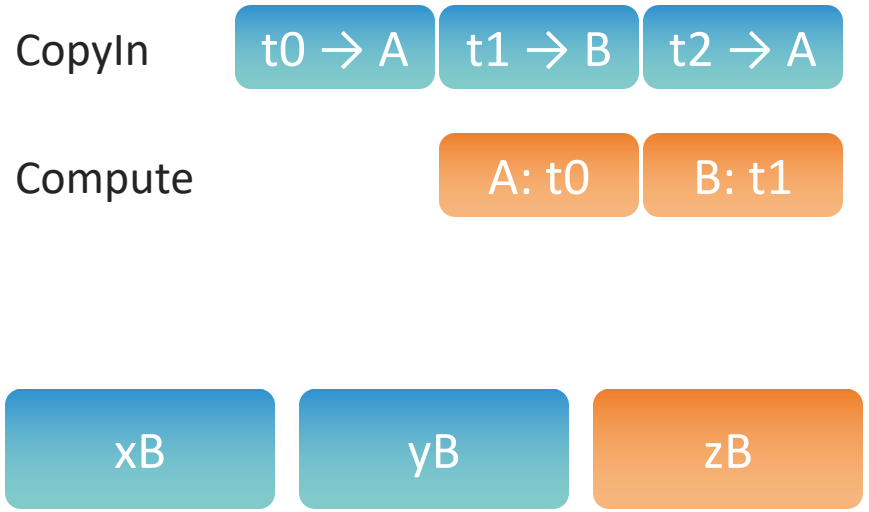
上海创智学院
Shanghai Innovation Institute

Input-side timeline; output transfer omitted to isolate buffer reuse

One input buffer: reuse waits



Two input buffers: A / B alternate



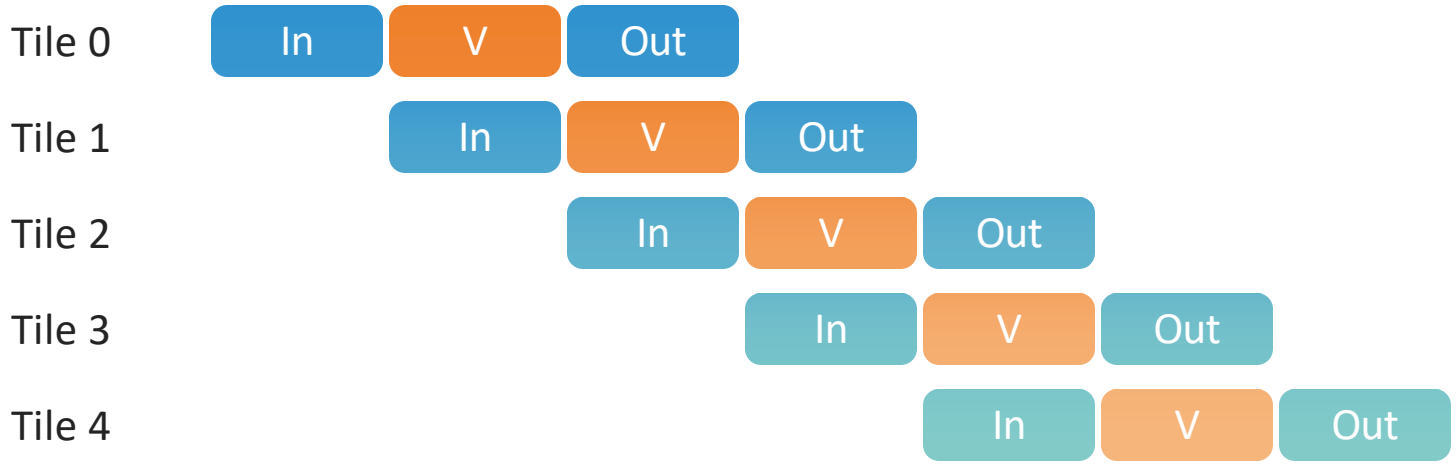
$$2 \times 3 \times 1024 \times 4 \text{ bytes} = 24 \text{ KiB}$$

The Slowest Stage Limits Throughput

上海创智学院
Shanghai Innovation Institute

Teaching model: 5 tiles, independent resources, sufficient buffers, no other overhead

Balanced: 1 / 1 / 1 µs



7 µs

15 / 7 ≈ 2.14×

Compute-bound: 1 / 3 / 1 µs



17 µs

$$5 + 4 \times 3 = 17\ \mu s$$

Regbase Reuses Values in Registers

上海创智学院 Shanghai Innovation Institute

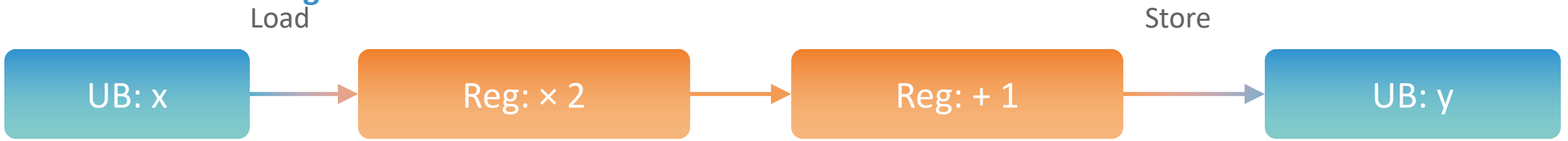
Example: y = 2x + 1 x = [1, 2, 3, 4]

Two operations with a UB intermediate



4 UB accesses per element in this baseline

One SIMD VF with register reuse



SIMD Masks the Final Vector

CANN 9.1 device-function excerpt Aligned UB allocations: at least 128 floats each

```
__simd_vf__ inline void add_one(
    __ubuf__ float* dst,
    __ubuf__ float* src, uint32_t count) {
  using namespace AscendC::Reg;
  RegTensor<float> x, y;
  constexpr uint32_t W =
      AscendC::GetVecLen() / sizeof(float);
  const uint32_t steps = (count + W - 1) / W;
  for (uint32_t r = 0; r < steps; ++r) {
    auto mask = UpdateMask<float>(count);
    LoadAlign(x, src + r * W);
    Adds(y, x, 1.0f, mask);
    StoreAlign(dst + r * W, y, mask);
  }
}
```

count = 70

FP32 lanes: W = 64

Round 0: 64 valid

Round 1

6 valid | 58 masked

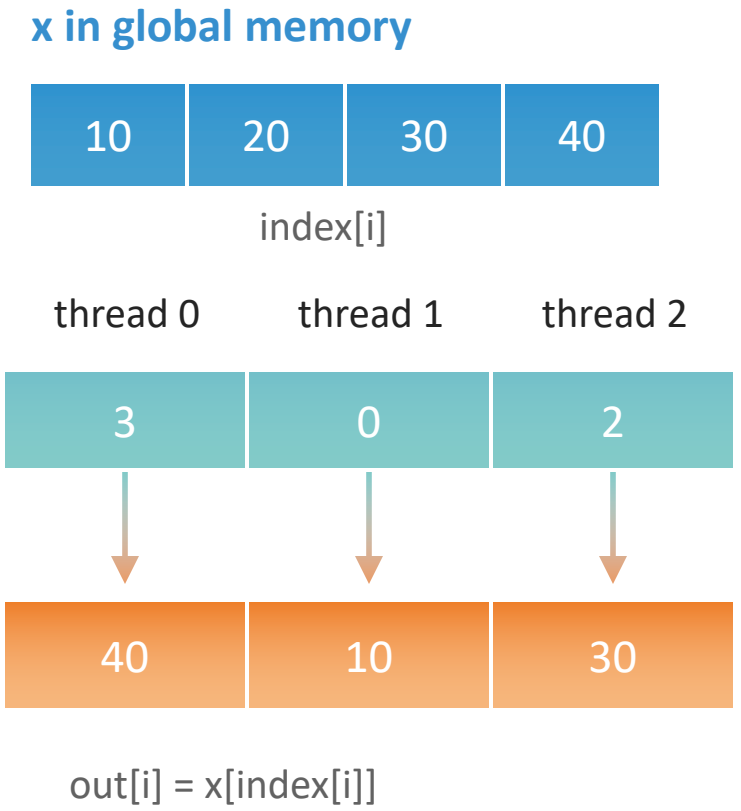
Only 6 values are stored

SIMT Gives Each Thread Its Own Index

上海创智学院 Shanghai Innovation Institute

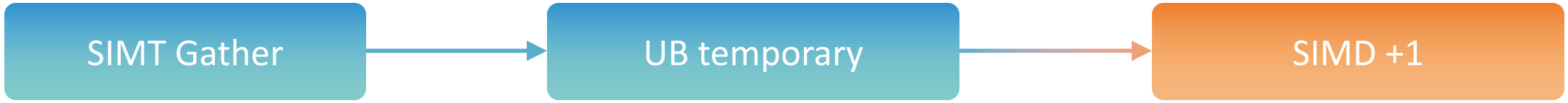
CANN 9.1 device-function excerpt One work block, 32 threads, 3 valid outputs

```
__simt_vf__ __launch_bounds__(32)
inline void gather(__gm__ float* x,
    __gm__ uint32_t* index,
    __ubuf__ float* out, uint32_t n) {
  uint32_t i = threadIdx.x;
  if (i < n) {
    out[i] = x[index[i]];
  }
}
```



SIMT and SIMD Exchange Data Through UB

Wrapper-kernel excerpt One block, 3 valid outputs, aligned UB allocations of 128 floats



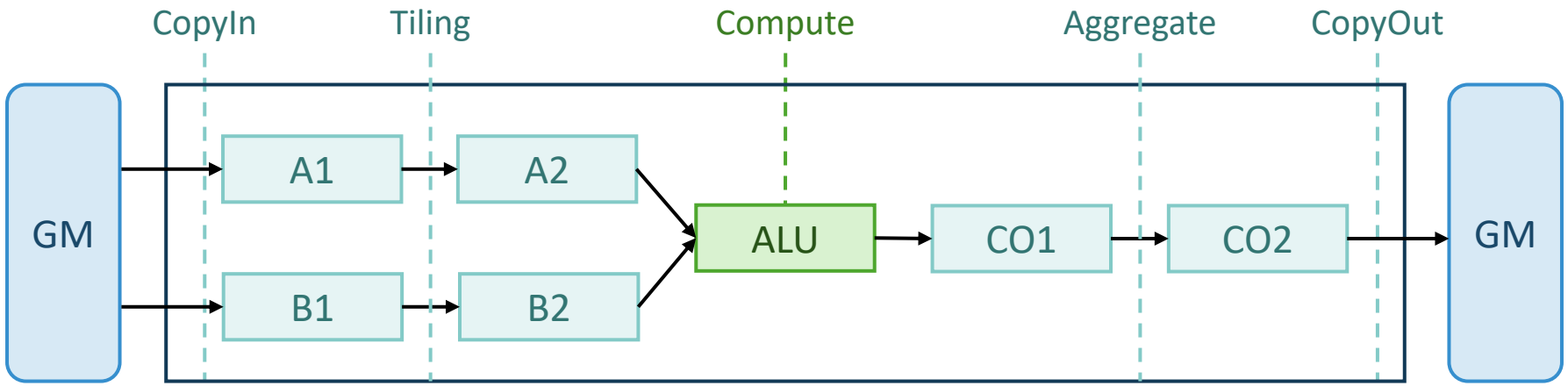
3-element illustration

[40, 10, 30]

[41, 11, 31]

```
asc_vf_call<gather>(dim3(32),
    xGmPtr, indexGmPtr, gatherUbPtr, 3);
asc_vf_call<add_one>(resultUbPtr, gatherUbPtr, 3);
AscendC::SetFlag<AscendC::HardEvent::V_MTE3>(0);
AscendC::WaitFlag<AscendC::HardEvent::V_MTE3>(0);
// GM writeback of 3 valid floats is omitted.
```

Ascend C Programming for AIC



A1: L1 buffer for matrix A
A2: L0A buffer for A tiles
B1: L1 buffer for matrix B
B2: L0B buffer for B tiles

CO1: Output buffer for tiled C
CO2: Output buffer for C

Pseudocode for AIC Programming

上海创智学院
Shanghai Innovation Institute

```
// Init
typedef MatmulType<TPosition::GM, CubeFormat::ND, half> aType;
typedef MatmulType<TPosition::GM, CubeFormat::ND, half> bType;
typedef MatmulType<TPosition::GM, CubeFormat::ND, float> cType;
typedef MatmulType<TPosition::GM, CubeFormat::ND, float> biasType;
Matmul<aType, bType, cType, biasType> mm;

REGIST_MATMUL_OBJ(&pipe, GetSysWorkSpacePtr(), mm, &tiling);

// CopyIn stage
mm.SetTensorA(gm_a);
mm.SetTensorB(gm_b);
mm.SetBias(gm_bias);

// Compute stage
while (mm.Iterate()) {
    // CopyOut Stage
    mm.GetTensorC(gm_c);
}
mm.End();
```

A Matmul Tile Accumulates Over K

Arithmetic illustration: A[2,4] × B[4,2], K-step = 2, zero-initialized C[2,2]

A

1	2	3	4
5	6	7	8

×

B

1	0
0	1
1	1
2	0

=

C

12	5
28	13

K = 0, 1

1	2
5	6

+

K = 2, 3

11	3
23	7

Accumulate in L0C

Keep partial sums until K is complete

Tile Layouts Change Memory Access

上海创智学院
Shanghai Innovation Institute

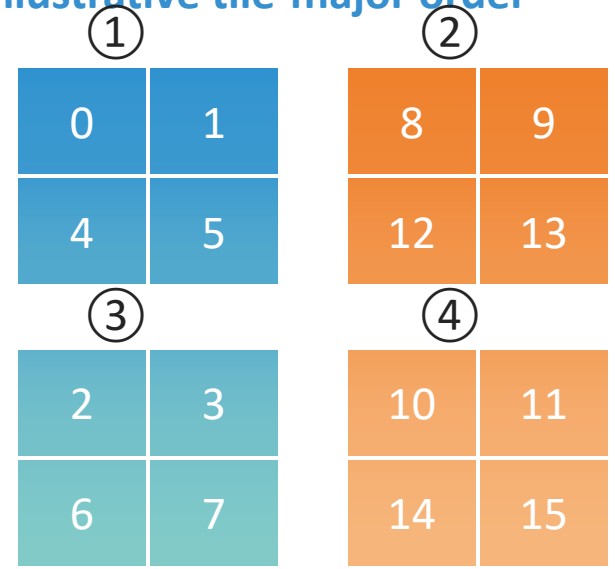
Same logical matrix; different physical order 2×2 blocks shown for illustration

Row-major ND

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

0 1 2 3 4 5 6 7 ...

Illustrative tile-major order



0 1 4 5 / 8 9 12 13 / ...

NDDMA Describes a Slice with Strides

上海创智学院 Shanghai Innovation Institute

CANN 9.1 CopyIn excerpt FP32 source [4,8] → UB tile [2,4]

0	1	2	3	4	5	6	7
8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23
24	25	26	27	28	29	30	31



10	11	12	13
18	19	20	21

Contiguous UB

Start = 1×8 + 2 = 10

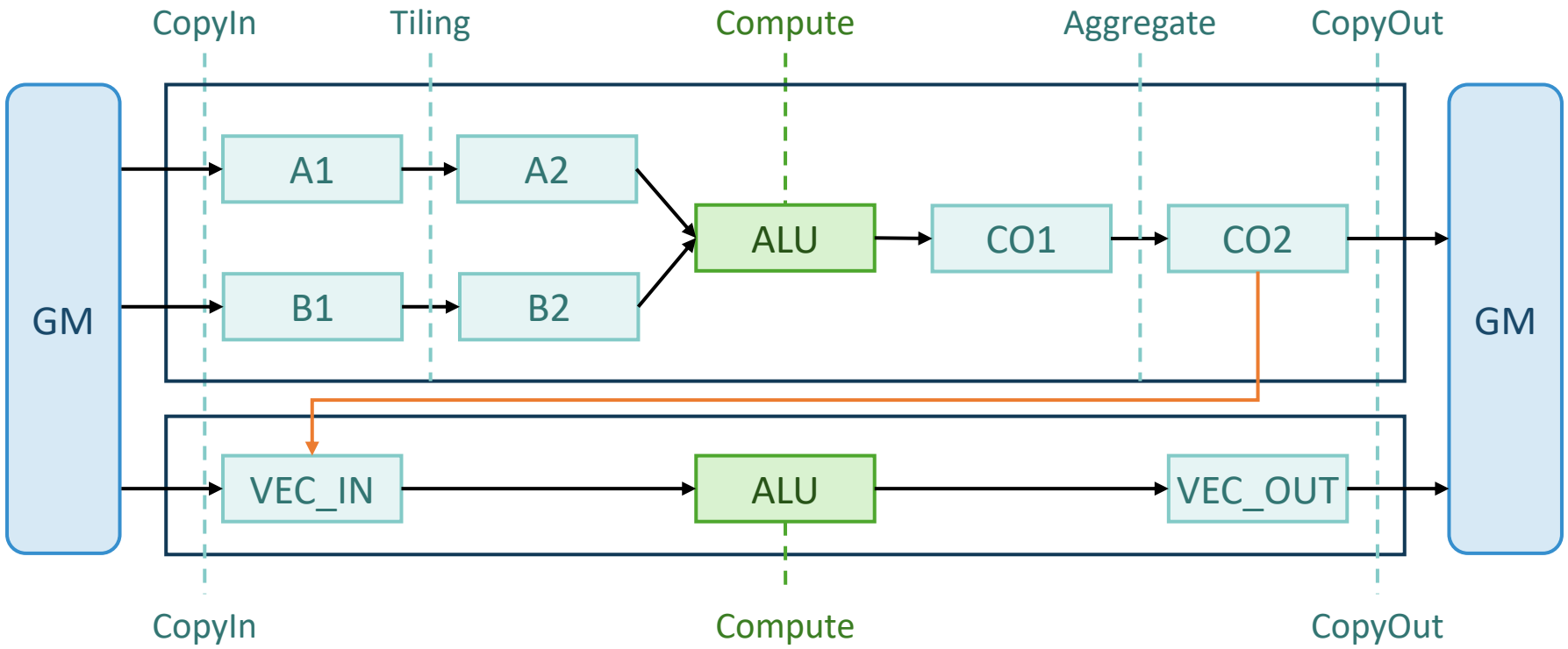
```
auto tile =
    inQueue.AllocTensor<float>();
AscendC::NdDmaLoopInfo<2> loops{
    {1, 8}, {1, 4}, {4, 2},
    {0, 0}, {0, 0}};
AscendC::NdDmaParams<float, 2>
    p{loops, 0};
AscendC::NdDmaDci();
AscendC::DataCopy<float, 2>(
    tile, src[10], p);
inQueue.EnQue(tile);
```

src strides: [1, 8]

dst strides: [1, 4]

size: [4, 2]

Ascend C Programming for Fused Operator



A1: L1 buffer for matrix A
A2: L0A buffer for A tiles
B1: L1 buffer for matrix B
B2: L0B buffer for B tiles

CO1: Output buffer for tiled C
CO2: Output buffer for C
VEC_IN: input buffer for AIV
VEC_OUT: output buffer for AIV

Pseudocode for Fused Operator

上海创智学院
Shanghai Innovation Institute

```
template<typename aType, typename bType, typename cType, typename biasType>
__aicore__ inline void MatmulLeakyKernel<aType, bType, cType, biasType>::Process() {
    // Step 1. Init
    REGIST_MATMUL_OBJ(&pipe, GetSysWorkSpacePtr(), matmulObj);
    matmulObj.Init(&tiling);
    matmulObj.SetTensorA(aGlobal);
    matmulObj.SetTensorB(bGlobal);
    matmulObj.SetBias(biasGlobal);

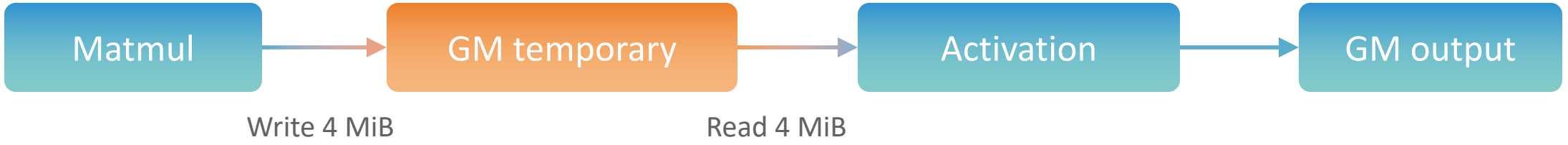
    // Step 2. iterate the tiling
    while (matmulObj.template Iterate<true>()) {
        // Step 3. Move the output to AIV
        reluOutLocal = reluOutQueue_.AllocTensor<cType>();
        matmulObj.template GetTensorC<true>(reluOutLocal, false, true);
       // Step 4. Compute on AIV
        AscendC::LeakyRelu(reluOutLocal, reluOutLocal, (cType)alpha, tiling.baseM * tiling.baseN);
        reluOutQueue_.EnQue(reluOutLocal);
        // Step 5. Write result back to GM
        reluOutQueue_.DeQue<cType>();
        ...
        AscendC::DataCopy(cGlobal[startOffset], reluOutLocal, copyParam);
        reluOutQueue_.FreeTensor(reluOutLocal);
    }
    matmulObj.End();
}
```

Fusion Removes Intermediate GM Traffic

上海创智学院 Shanghai Innovation Institute

Matmul output: 1024×1024 FP32 values Count only the intermediate tensor

Separate kernels



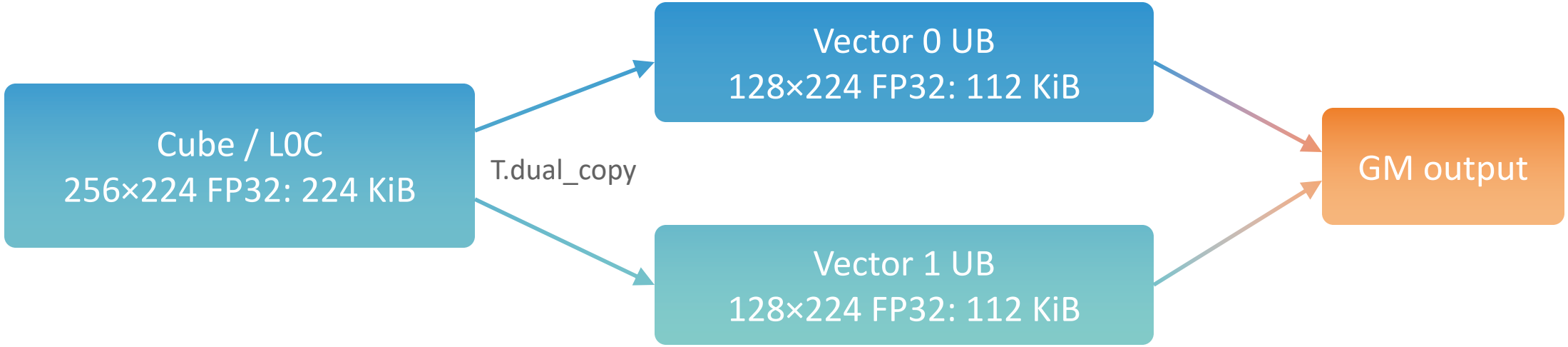
Fused, one tile at a time



$$\text{Saved traffic} = 2 \times 1024 \times 1024 \times 4\ \text{bytes} = 8\ \text{MiB}$$

950 Transfers Cube Results Directly to UB

One 256×224 FP32 result tile; the illustrated epilogue is ReLU (α = 0)




```
T.dual_copy(C_l0c, C_ub)
with T.SimtVF(threads=128):
    for i, j in T.Parallel(128, 224):
        C_ub[i, j] = T.max(C_ub[i, j], 0)
```

TileLang Expresses a Fused Cube–Vector Kernel

上海创智学院 Shanghai Innovation Institute

Complete Python example Ascend 950 + compatible CANN / torch_npu / TileLang

```python
import torch, torch_npu, tilelang
import tilelang.ascend.language as T

@tilelang.jit(target="ascend")
def gemm_relu(A, B):
    A: T.Tensor((256, 128), T.bfloat16)
    B: T.Tensor((224, 128), T.bfloat16)
    C = T.empty((256, 224), T.float32)
    with T.Kernel(1):
        a = T.alloc_l1((256, 128), T.bfloat16)
        b = T.alloc_l1((224, 128), T.bfloat16)
        c = T.alloc_l0c((256, 224), T.float32)
        v = T.alloc_shared(
            (128, 224), T.float32)
        T.copy(A, a)
        T.copy(B, b)
        T.gemm(a, b, c, transpose_B=True,
               clear_accum=True)
        T.dual_copy(c, v)
        with T.SimtVF(threads=128):
            for i, j in T.Parallel(128, 224):
                v[i, j] = T.max(v[i, j], 0)
        T.dual_copy(v, C)
    return C

a = torch.randn(256, 128, device="npu",
                dtype=torch.bfloat16)
b = torch.randn(224, 128, device="npu",
                dtype=torch.bfloat16)
c = gemm_relu(a, b)
ref = torch.relu(a.float() @ b.float().T)
torch.testing.assert_close(
    c, ref, rtol=1e-2, atol=1e-2)
```

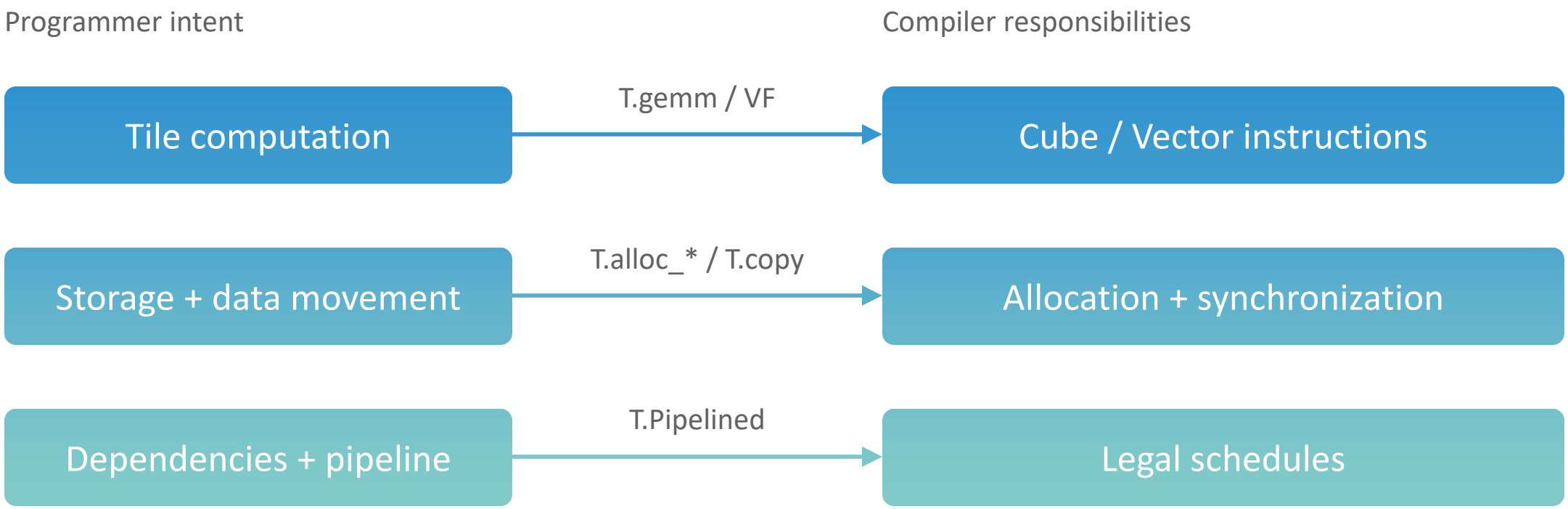
Programming Paradigm

上海创智学院
Shanghai Innovation Institute

- **Utilize hardware by applying instruction pipeline**
- Each stage of tasks:
  - Allocate local memory or get local memory from upstream queue
  - Compute or copy data
  - Push processed data to downstream queue
  - Free useless local memory

Compilers Organize Tiles and Dependencies





Appendix A: The Processing Flow of EnQue / DeQue

上海创智学院 Shanghai Innovation Institute

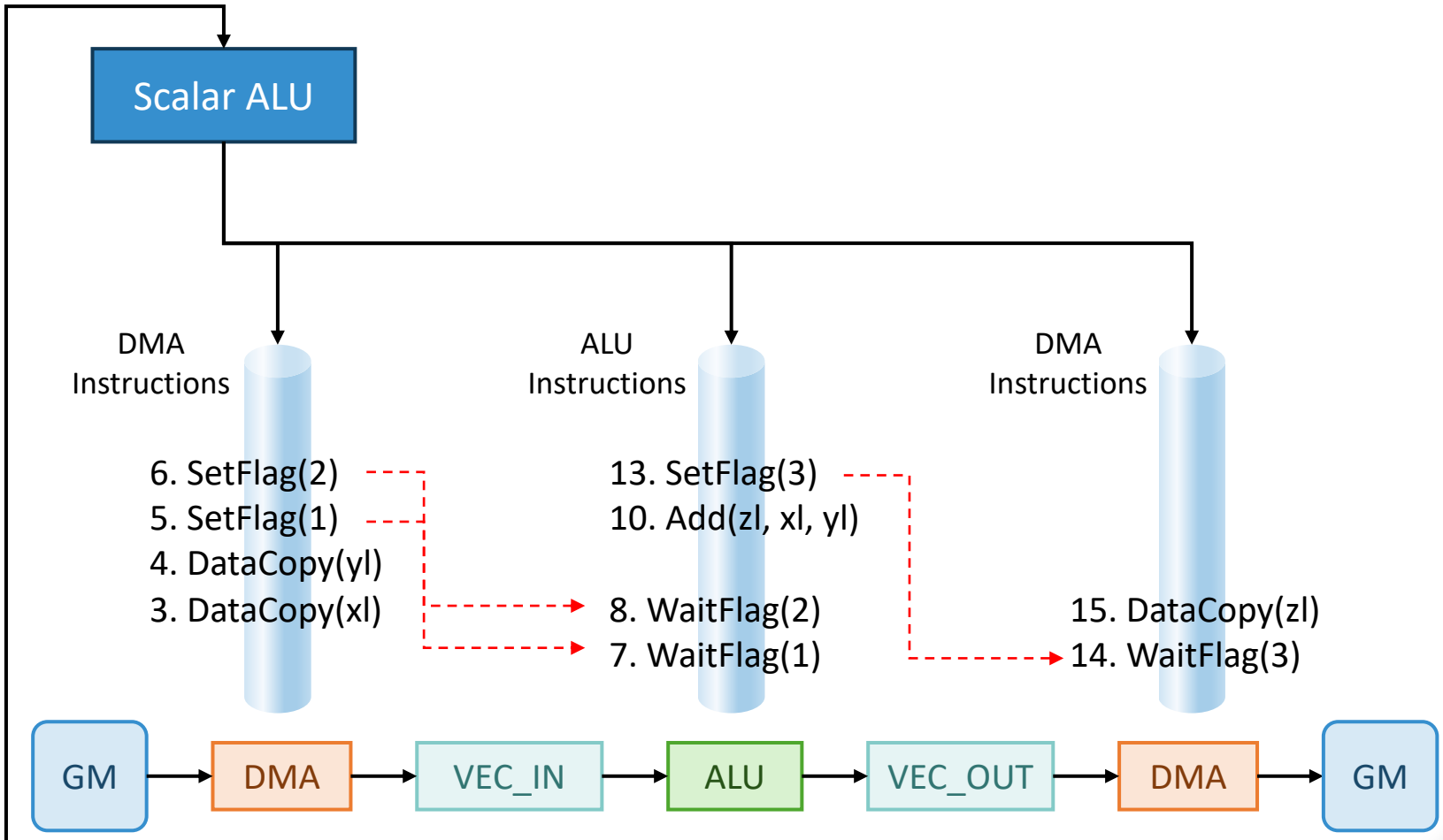
1. Scalar unit read the instruction
2. The instructions are dispatched to the corresponding unit's queues
3. The various execution units execute these instruction in parallel.
4. EnQue/DeQue resolves the **Read-After-Write (RAW) hazard** for memory:
    1. An **EnQue** call issues a synchronization instruction (**set**), sending a signal to activate the wait.
    2. A **DeQue** call issues a synchronization instruction (**wait**), waiting for the data write to complete.
    3. The wait must wait for the set signal before it can execute; otherwise, it is blocked.

Appendix A: The Processing Flow of EnQue / DeQue

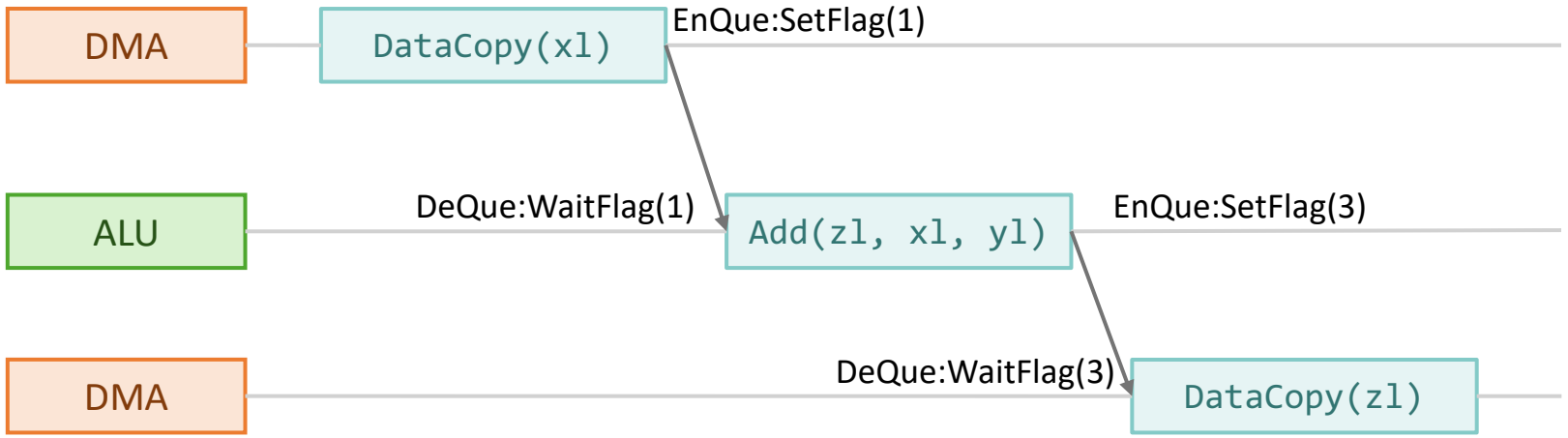
上海创智学院 Shanghai Innovation Institute

Operator Instruction Sequence

```
16. z.FreeTensor(zl)
15. DataCopy(zl)
14. zl = z.DeQue()
13. z.EnQue(zl)
12. y.FreeTensor(yl)
11. x.FreeTensor(xl)
10. Add (zl, xl, yl)
9.  zl = z.AllocTensor()
8.  yl = y.DeQue()
7.  xl = x.DeQue()
6.  y.EnQue(yl)
5.  x.EnQue(xl)
4.  DataCopy(yl)
3.  DataCopy(xl)
2.  yl = y.AllocTensor()
1.  xl = x.AllocTensor()
```



Appendix A: The Processing Flow of EnQue / DeQue



Appendix B: The Processing Flow of AllocTensor / FreeTensor

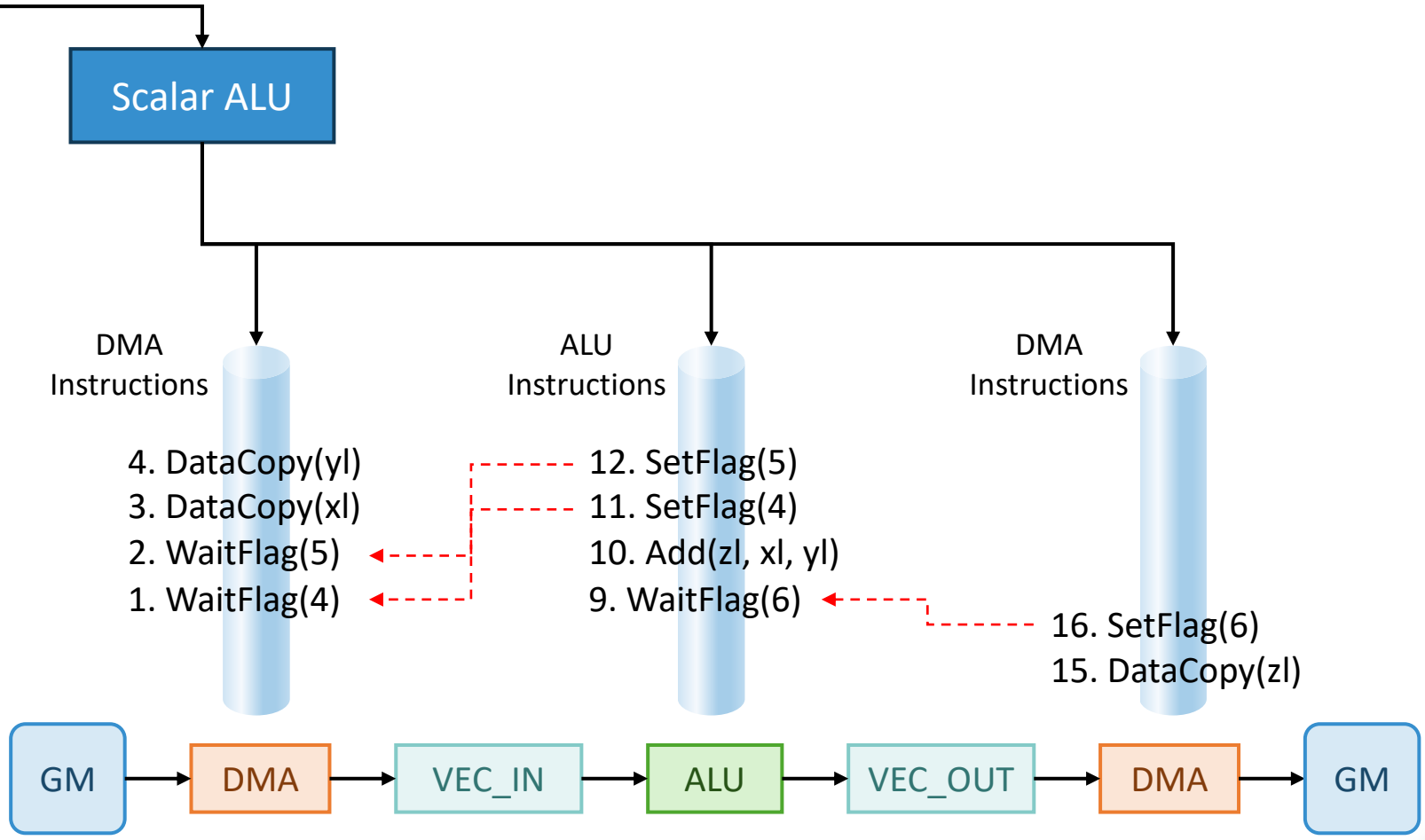
1. Scalar unit read the instruction
2. The instructions are dispatched to the corresponding unit's queues
3. The various execution units execute these instruction in parallel.
4. AllocTensor/FreeTensor resolves the **Write-After-Read (WAR)** hazard:
    1. An **AllocTensor** call issues a synchronization instruction (**wait**), waiting for the memory to finish being read.
    2. A **FreeTensor** call issues a synchronization instruction (**set**), signaling that the memory is released and can be repeatedly written to.
    3. The wait must wait for the set signal before it can execute; otherwise, it is blocked.

Appendix B: The Processing Flow of AllocTensor / FreeTensor

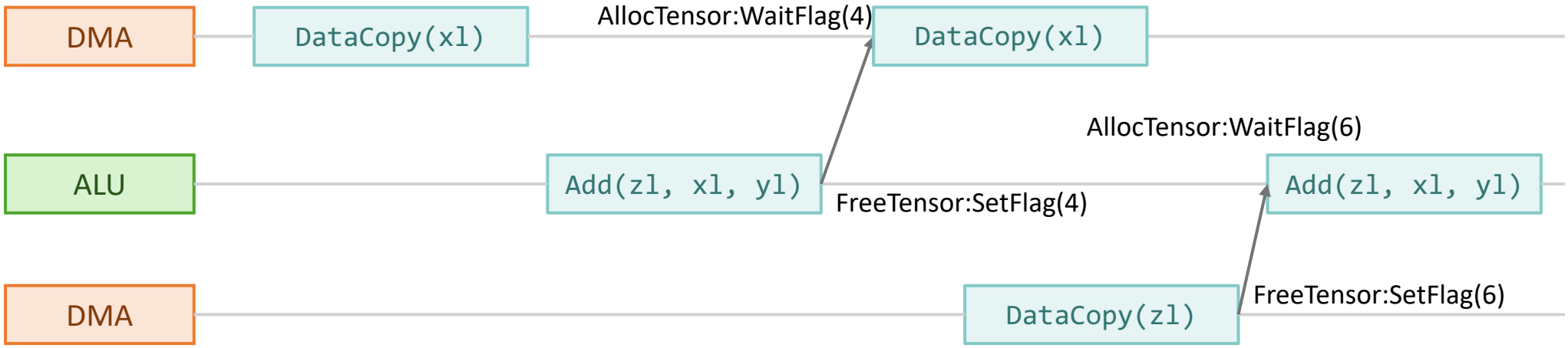
上海创智学院 Shanghai Innovation Institute

Operator Instruction Sequence

```
16. z.FreeTensor(zl)
15. DataCopy(zl)
14. zl = z.DeQue()
13. z.EnQue(zl)
12. y.FreeTensor(yl)
11. x.FreeTensor(xl)
10. Add (zl, xl, yl)
9.  zl = z.AllocTensor()
8.  yl = y.DeQue()
7.  xl = x.DeQue()
6.  y.EnQue(yl)
5.  x.EnQue(xl)
4.  DataCopy(yl)
3.  DataCopy(xl)
2.  yl = y.AllocTensor()
1.  xl = x.AllocTensor()
```



Appendix B: The Processing Flow of AllocTensor / FreeTensor



Acknowledgement

The development of this course, including its structure, content, and accompanying presentation slides, has been significantly influenced and inspired by the excellent work of instructors and institutions who have shared their materials openly. We wish to extend our sincere acknowledgement and gratitude to the following courses, which served as invaluable references and a source of pedagogical inspiration:

- Machine Learning Systems[15-442/15-642], by **Tianqi Chen** and **Zhihao Jia** at **CMU**.
- Advanced Topics in Machine Learning (Systems)[CS6216], by **Yao Lu** at **NUS**

While these materials provided a foundational blueprint and a wealth of insightful examples, all content herein has been adapted, modified, and curated to meet the specific learning objectives of our curriculum. Any errors, omissions, or shortcomings found in these course materials are entirely our own responsibility. We are profoundly grateful for the contributions of the educators listed above, whose dedication to teaching and knowledge-sharing has made the creation of this course possible.

System for Artificial Intelligence

Thanks

Siyuan Feng
Shanghai Innovation Institute